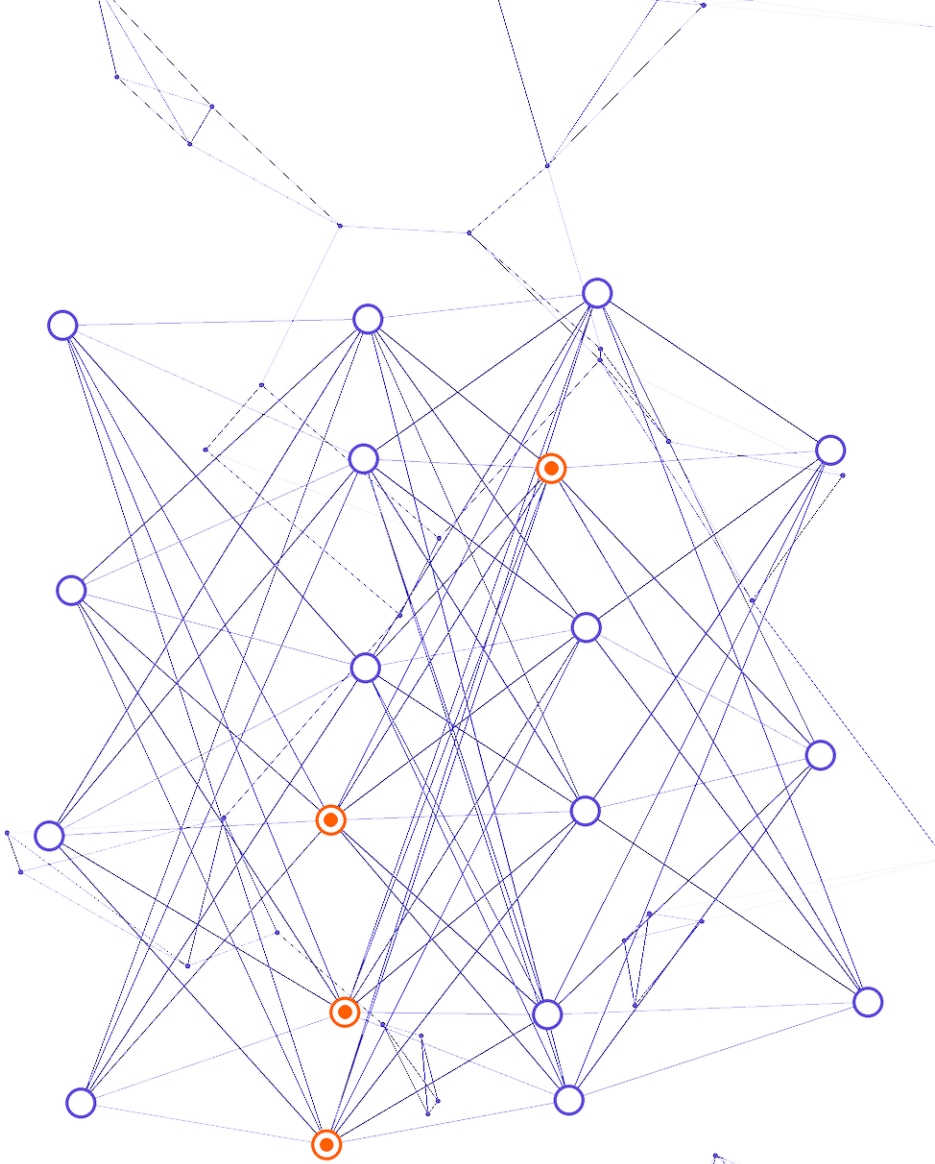
04

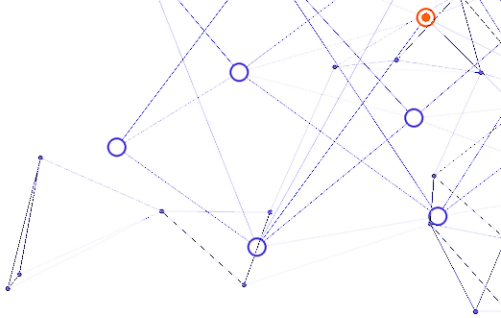
Regressão linear e gradiente descendente

Hipótese, função de custo e como encontrar os melhores parâmetros



C.E.S.A.R school

Notação

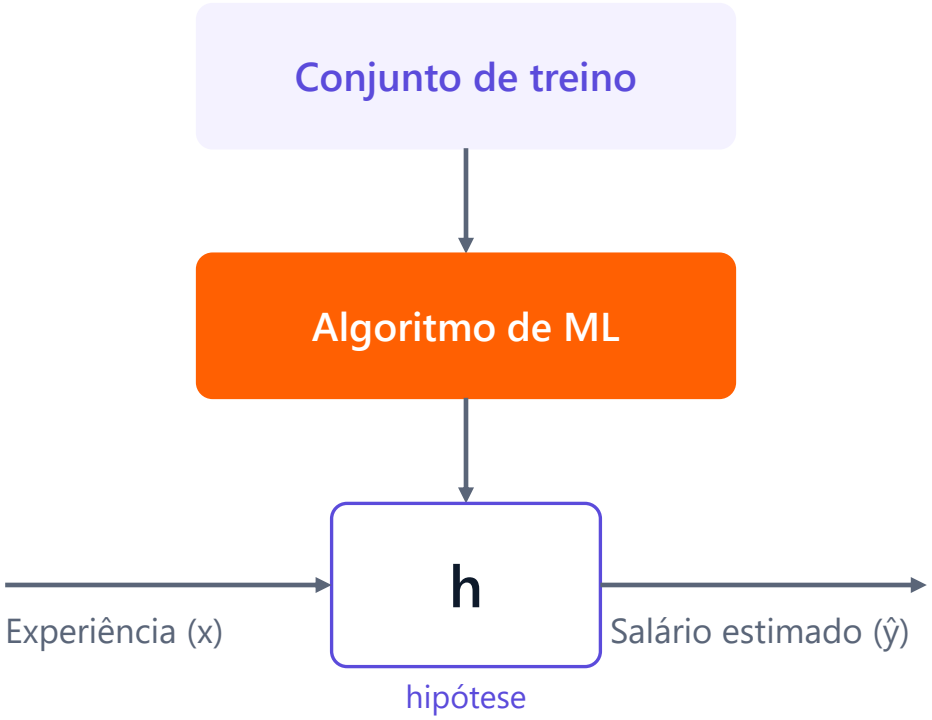


m	número de exemplos de treinamento
x	variáveis de entrada (features)
y	variável de saída (target)
$(x^{(i)}, y^{(i)})$	i-ésimo exemplo de treino

Experiência (anos) x	Salário (k R$) y
2	36
8	50
13	72
20	98
...	...

Fontes: Andrew Ng, Machine Learning (Coursera)

Como o modelo é treinado



A hipótese h mapeia x em ŷ

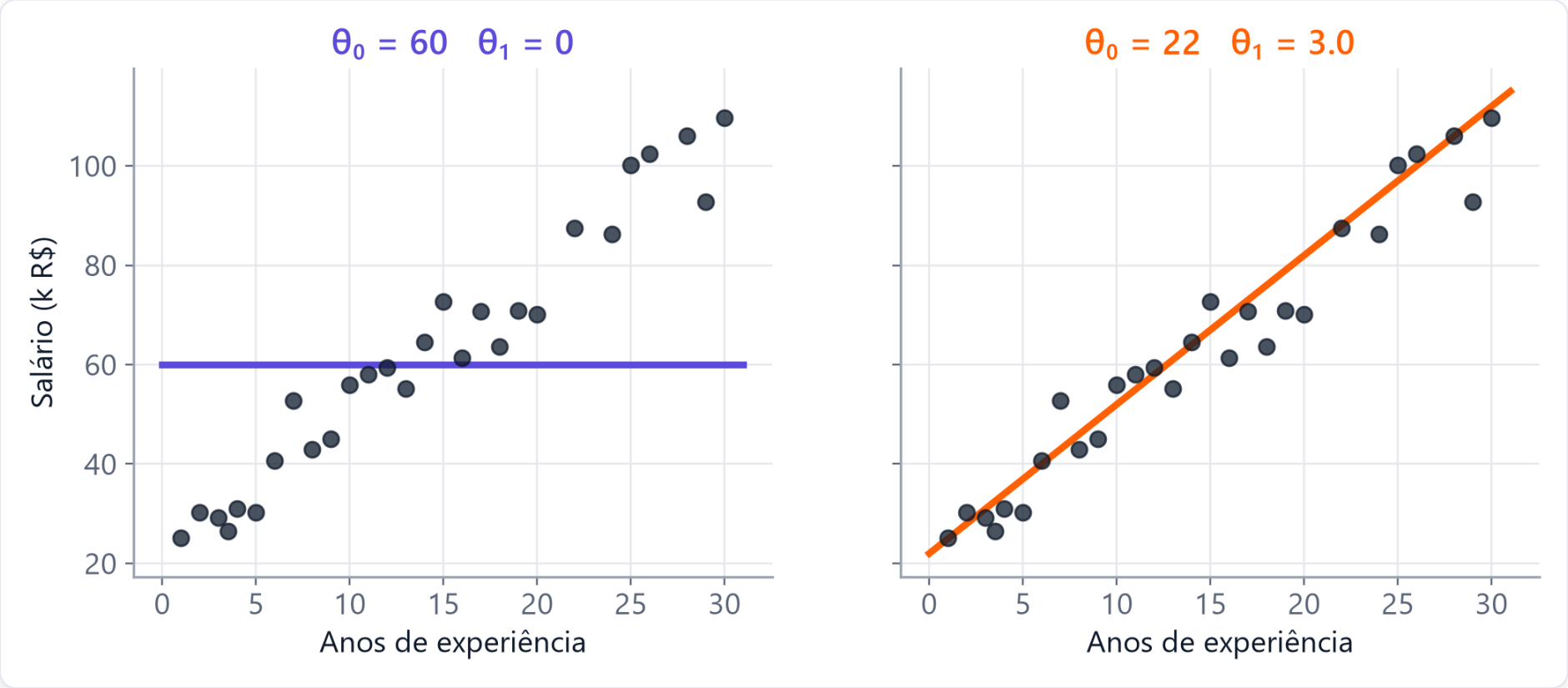
$$h\theta(x) = \theta_0 + \theta_1 x$$

- **θ_0** intercepto (bias): salário de quem tem zero anos de experiência
- **θ_1** inclinação (peso): quanto o salário cresce por ano
- Treinar = escolher θ_0 e θ_1 para que h(x) fique perto de y nos exemplos de treino

Fontes: Andrew Ng, Machine Learning (Coursera)

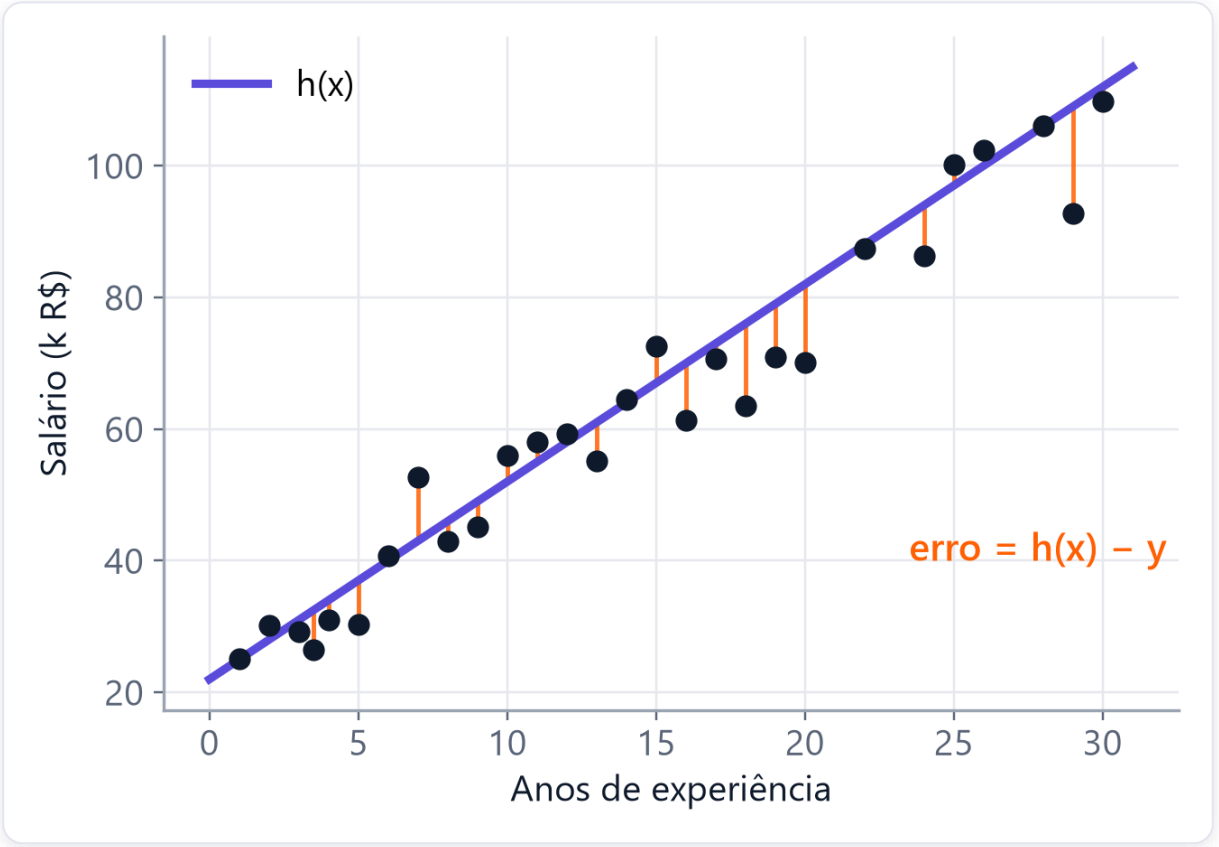
Qual hipótese é melhor?

$$h\theta(x) = \theta_0 + \theta_1 x$$



Fontes: Andrew Ng, Machine Learning (Coursera)

Medindo o erro: Mean Squared Error



Escolher θ_0, θ_1 para que h(x) fique perto de y em todo o conjunto de treino

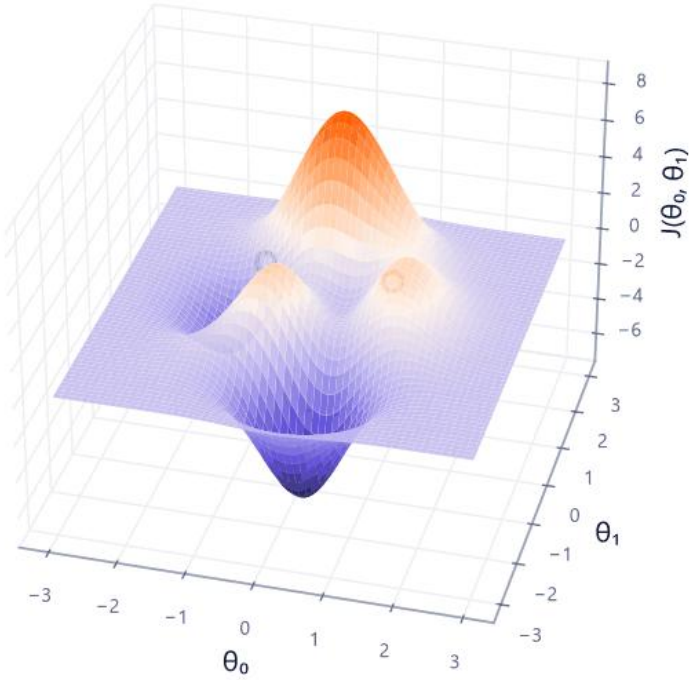
$$J(\theta_0, \theta_1) = {}^{1}/_{2m} \Sigma_i \left(h\theta(x^{(i)}) - y^{(i)} \right)^2$$

Objetivo: minimizar $J(\theta_0, \theta_1)$

Elevar ao quadrado penaliza mais os erros grandes e torna J suave e convexa na regressão linear.

Fontes: Andrew Ng, Machine Learning (Coursera)

A paisagem da função de custo



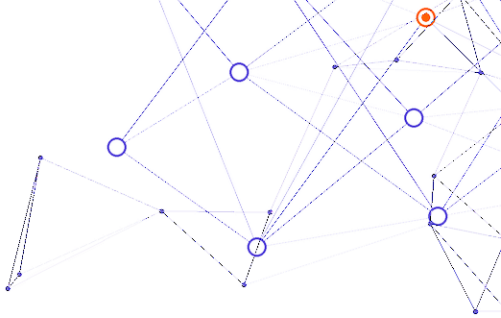
Cada ponto da superfície é um par (θ_0, θ_1) e o custo J correspondente.

Para descer, damos passos na direção oposta ao gradiente — a direção de maior inclinação.

Em funções não convexas o ponto de partida importa: o caminho laranja chega ao mínimo global; o preto fica preso num mínimo local.

Fontes: Andrew Ng, Machine Learning (Coursera)

O algoritmo



Objetivo

Encontrar θ_0 e θ_1 que minimizam $J(\theta_0, \theta_1)$

Como fazer

- Começar com θ_0 e θ_1 quaisquer
- Calcular o gradiente de J nesse ponto
- Dar um passo na direção oposta, de tamanho α
- Repetir até que J pare de diminuir

repita até convergir:

$$\theta_j := \theta_j - \alpha \cdot \partial J(\theta_0, \theta_1) \,/\, \partial \theta_j$$

atualizando j = 0 e j = 1 simultaneamente

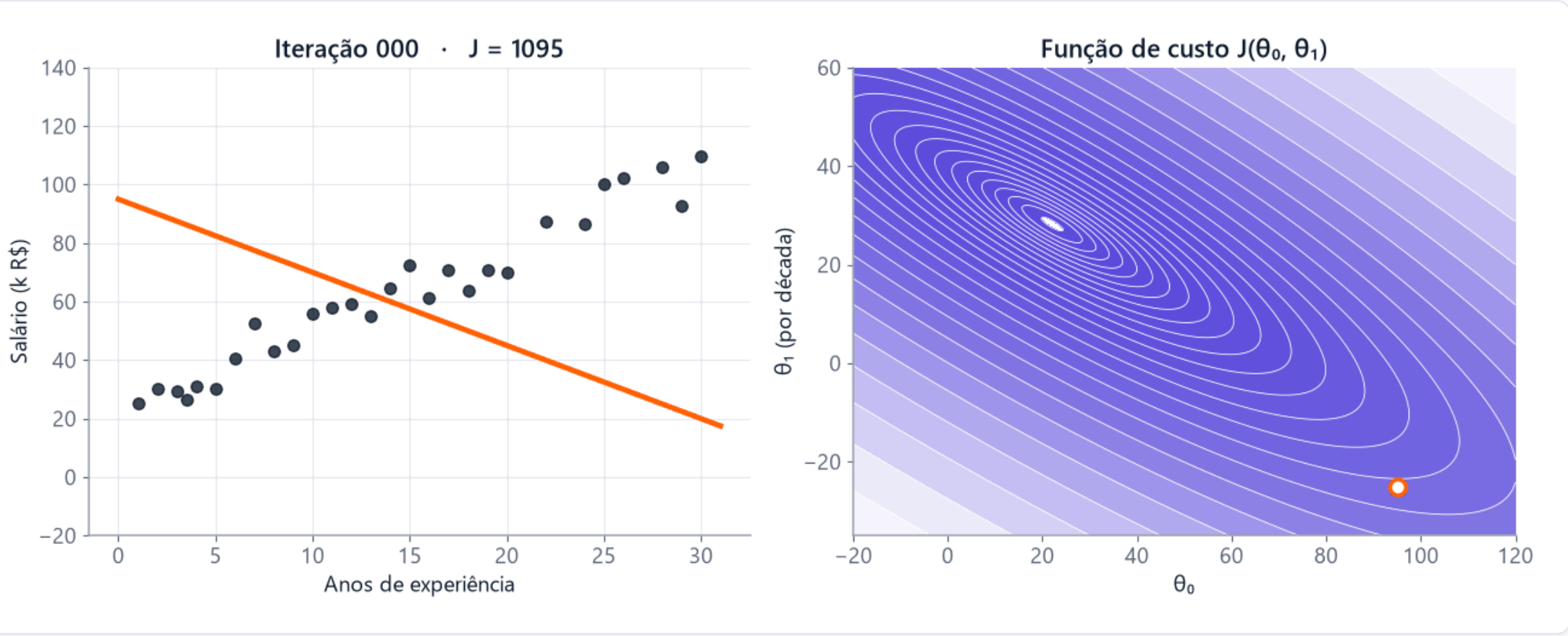
Para a regressão linear:

$$\partial J/\partial \theta_0 = {}^{1}/_{m} \, \Sigma_i \, (h\theta(x^{(i)}) - y^{(i)})$$
$$\partial J/\partial \theta_1 = {}^{1}/_{m} \, \Sigma_i \, (h\theta(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

α = taxa de aprendizado (learning rate)

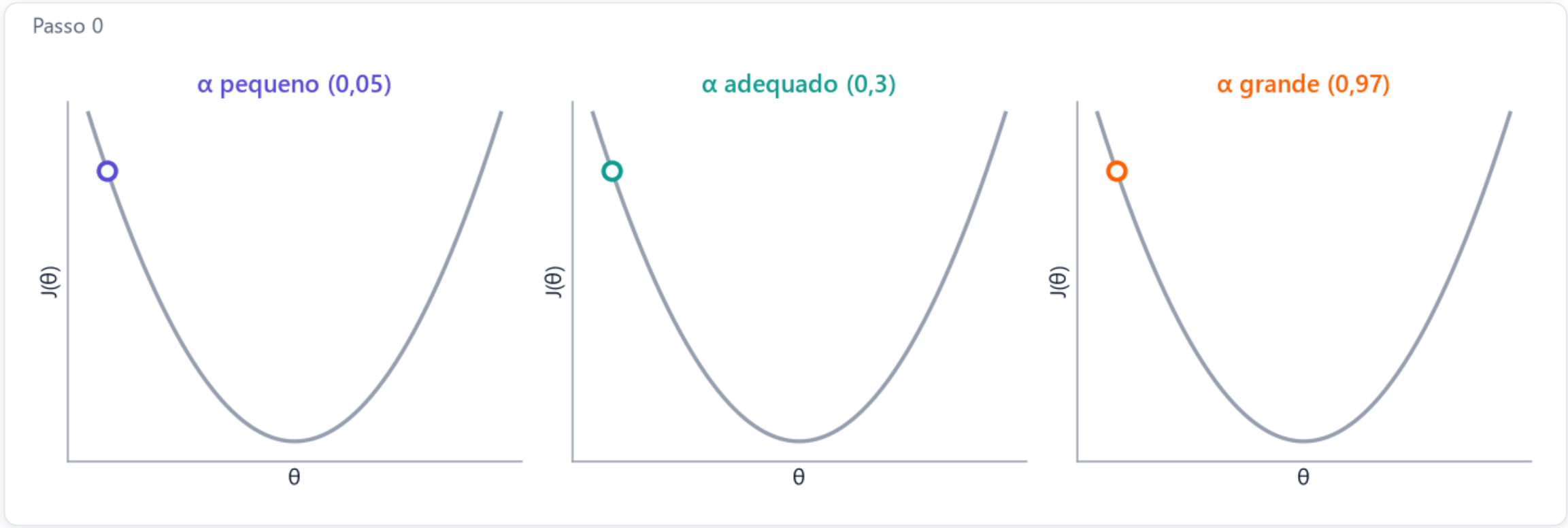
Fontes: Andrew Ng, Machine Learning (Coursera)

Ajustando a reta, passo a passo



À esquerda, a reta h(x) se ajusta aos dados. **À direita,** o mesmo processo visto nas curvas de nível de J: cada passo desce em direção ao mínimo.

A escolha da taxa de aprendizado α



α pequeno converge, mas devagar: muitos passos

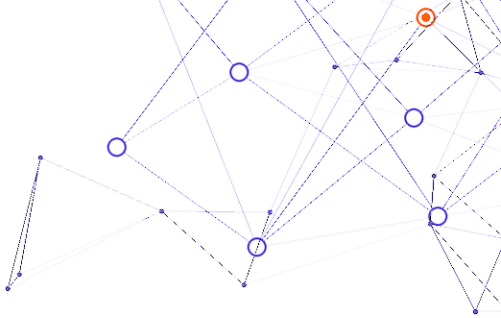
α adequado converge rápido e de forma estável

α grande oscila em torno do mínimo ou diverge

Fontes: Andrew Ng, Machine Learning (Coursera)

AULAS PRÁTICAS · NOTEBOOKS 2.X

Regressão linear: do NumPy ao PyTorch



- 2.1 Demo: ajuste interativo de uma reta com gradiente descendente
- 2.2 Regressão linear do zero em NumPy (gradiente e solução analítica)
- 2.3 Reescrevendo com tensores do PyTorch
- 2.4 Autograd e nn.Linear
- 2.5 nn.MSELoss e optim.SGD
- 2.6 Dataset real de salários (Kaggle): treino × validação

Nos notebooks: $\hat{y} = w_0 + w_1 x \quad (w_0 = \theta_0,\ w_1 = \theta_1)$

```
model = nn.Linear(1, 1)          # h(x) = θ₀ + θ₁x
criterion = nn.MSELoss()         # J(θ)
optimizer = optim.SGD(model.parameters(), lr=0.3)

for epoch in range(120):
    y_hat = model(x_train)       # forward
    loss = criterion(y_hat, y_train)
    optimizer.zero_grad()
    loss.backward()              # autograd: ∇J
    optimizer.step()             # θ := θ − α∇J
```

Fontes: Repositório da disciplina (branch 2026.01-A) · PyTorch: nn.MSELoss